

Unit 5: Device Management

5.1 Principle of I/O Hardware:

- I/O Devices
- Device Controllers
- Direct Memory Access

5.2 Principle of I/O Software:

- Goals of I/O Software
- Interrupt Handlers
- Device Drivers

5.3 Disk Management:

- Disk Structure
- Disk Scheduling Algorithm
- Error Handling and Formatting
- RAID
- Stable Storage Management

5.4 Terminals:

- Terminal Hardware
- Memory-Mapped Terminals
- Input/output Software

I/O Management

- The two main jobs of a computer are I/O and processing. In many cases, the main job is I/O and the processing is merely incidental. For instance, when we browse a web page or edit a file, our immediate interest is to read or enter some information, not to compute an answer.
- All computers have physical devices for acquiring input and producing output.
- The OS is responsible to manage and control all the I/O operations and I/O devices.

Hardware Organization

- The I/O devices, memory, and the CPU communicate with each other by way of one or more communication buses.

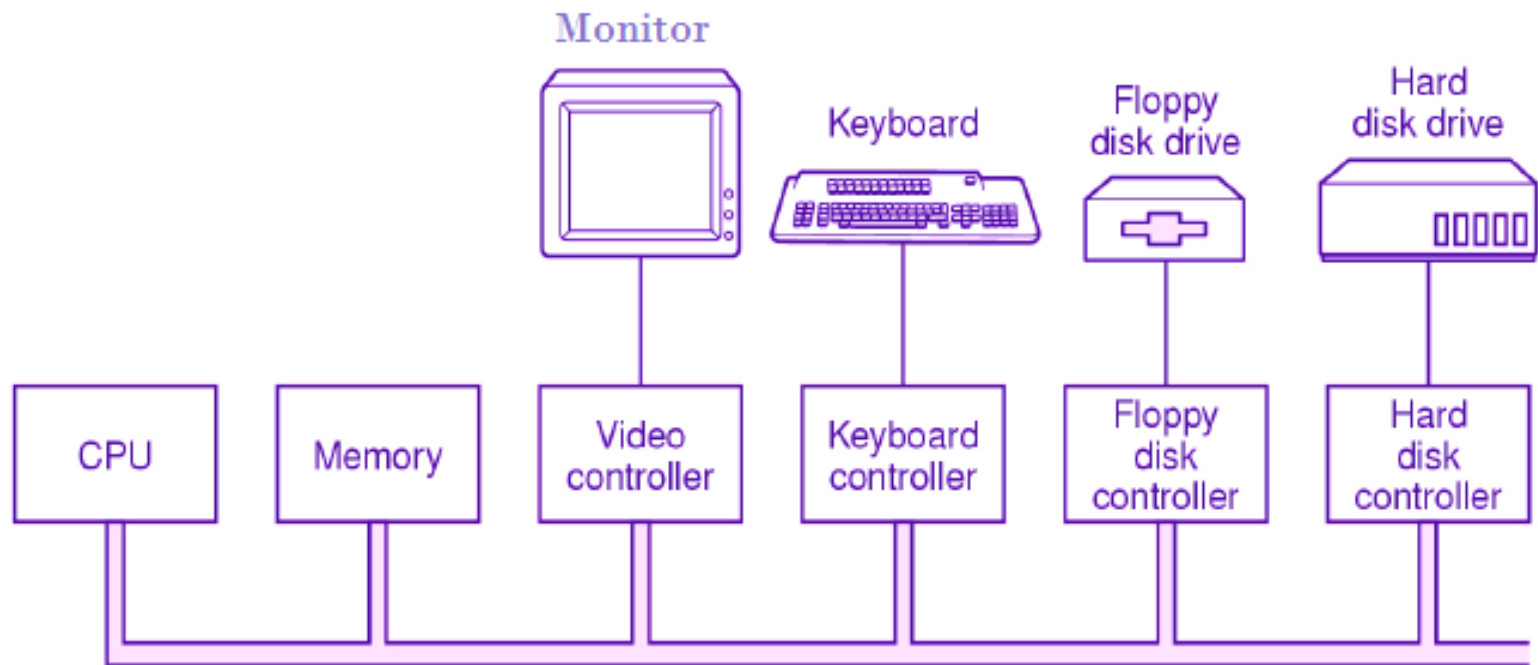


Fig: Single-Bus Architecture

I/O Devices

- The I/O units which consist mechanical components are called I/O devices such as hard-disk drive, printer, etc.
- There are roughly two types of I/O devices: **block devices** and **character devices**.
- **Block devices:** A block device is one that stores information in fixed-sized blocks, each one with its own address. Common block sizes range from 512 bytes to 32768 bytes. All transfers are in units of one or more (consecutive) blocks. The essential property of a block device is that it is possible to read or write each block independently of all the other ones i.e. direct access. Example: hard disks, CDROMs, USB sticks, etc.
- **Character devices:** A character device delivers or accepts a stream of characters, without regard to any block structure. It is not addressable and does not have any seek operation. Printers, network interfaces, mice (for pointing), rats (for psychology lab experiments), and most other devices that are not disk-like are character devices.
- This classification scheme is not perfect. Some devices are neither block nor character such as clock. Clocks are neither block addressable nor do they generate or accept character streams. All they do is cause interrupts at well-defined intervals.

Device Controllers

- A controller is a collection of electronics that can operate a port, a bus, or a device. For example, a serial port controller in the computer controls the signals on the wires of a serial port. On personal computers, it often takes the form of a chip on the parentboard or a printed circuit card that can be inserted into a (PCI) expansion slot.
- A single controller can handle multiple identical devices.
- Some devices have their own built-in controller. If we look at a disk drive, we will see a circuit board attached to one side. This board is the disk controller.
- How can the processor give commands and data to a controller to accomplish an I/O transfer? The answer is that the controller has one or more registers for data and control signals. The processor communicates with the controller by reading and writing bit patterns in these registers.
- For example: When transferring a disk block of size 512 bytes, the block is first assembled, bit by bit, in a buffer inside the controller. After its checksum has been verified and the block has been declared to be error free, it can then be copied to main memory.

Memory-Mapped I/O

- Each controller has a few **registers** that are used for communicating with the CPU. By writing into these registers, the operating system can command the device to deliver data, accept data, switch itself on or off, or otherwise perform some action. By reading from these registers, the operating system can learn what the device's state is, whether it is prepared to accept a new command, and so on.
- In addition to the **control registers**, many devices have a **data buffer** that the operating system can read and write. For example, a common way for computers to display pixels on the screen is to have a video RAM, which is basically just a data buffer, available for programs or the operating system to write into.
- **Issue:** How the CPU communicates with the control registers and the device data buffers? => Two approaches are used: (a) *Separate I/O and memory space* (b) *Memory-mapped I/O*

Memory-Mapped I/O...

- In the first approach, each control register is assigned an **I/O port** number, an 8- or 16-bit integer. Using a special I/O instruction such as

IN REG,PORT

the CPU can read in control register PORT and store the result in CPU register REG. Similarly, using

OUT PORT,REG

the CPU can write the contents of REG to a control register.

- In this scheme, the address spaces for memory and I/O are different, as shown in figure (a) below. The instructions

IN Ro,4

and

MOV Ro,4

are completely different in this design. The former reads the contents of I/O port 4 and puts it in Ro whereas the latter reads the contents of memory word 4 and puts it in Ro. The 4s in these examples thus refer to different and unrelated address spaces.

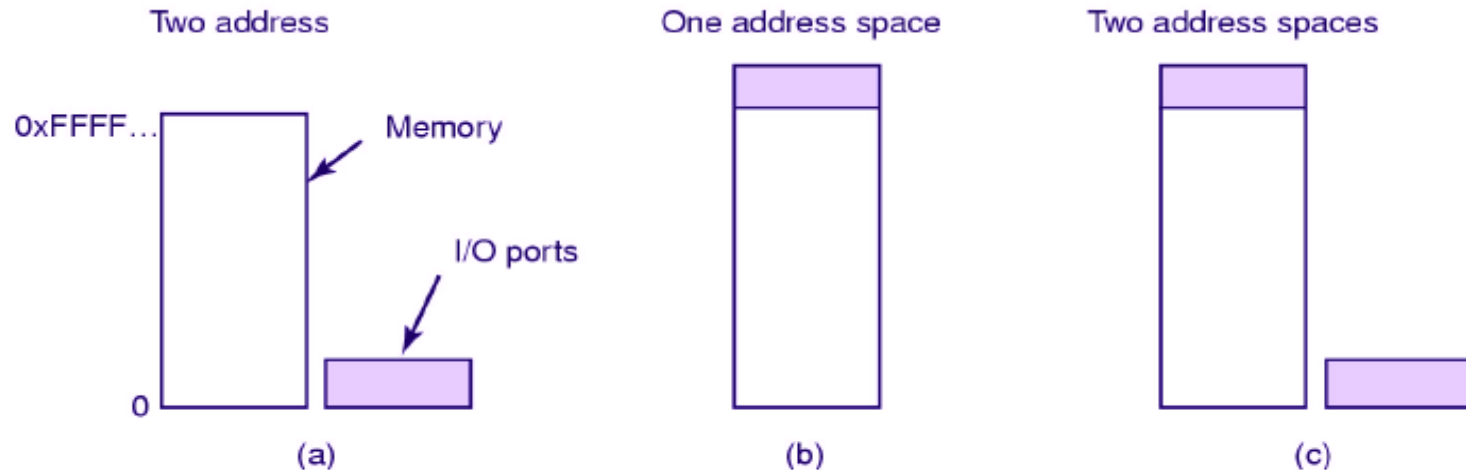


Figure: (a) Separate I/O and memory space (b) Memory-mapped I/O (c) Hybrid

- In the second approach, all the control registers are mapped into the memory space, as shown in figure (b) above. Each control register is assigned a unique memory address to which no memory is assigned. This system is called **memory-mapped I/O**. Usually, the assigned addresses are at the top of the address space. A hybrid scheme, with memory-mapped I/O data buffers and separate I/O ports for the control registers is shown in figure (c). The Pentium uses this architecture, with addresses 640K to 1M being reserved for device data buffers in IBM PC compatibles, in addition to I/O ports 0 through 64K.

Memory-Mapped I/O...

How do these schemes work?

- When a CPU wants to read a word, either from memory or from an I/O port, it puts the address it needs on the bus' address lines and then asserts the *READ* signal on a bus' control line. A second signal line is used to tell whether I/O or memory space is needed. If it is memory space, the memory responds to the request. If it is I/O space, the I/O device responds to the request.

Memory-Mapped I/O...

Advantages:

- As special I/O instructions are needed to read and write the device control registers, access to them requires the use of assembly code since there is no way to execute an IN or OUT instruction in C or C++. However, with memory mapped I/O, the device control registers are just variables in memory and can be easily addressed in C just like other variables. Hence, with memory mapped I/O, an I/O device driver can be written entirely in C.
- No special protection mechanism is needed to keep user processes from performing I/O. The OS just has to avoid from putting that portion of the address space containing the control registers in any user's virtual address space.
- Every instruction that can reference memory can also reference control registers (in hybrid). For example, if there is an instruction, TEST, that tests a memory word for 0, it can also be used to test a control register for 0, which might be the signal that the device is idle and can accept a new command. The assembly language code might look like this:

```
LOOP: TEST PORT_4      // check if port 4 is 0
      BEQ READY        // if it is 0 go to ready
      BRANCH LOOP      // otherwise, continue testing
```

READY:

If memory-mapped I/O is not present, the control register must first be read into the CPU, then tested, requiring two instructions instead of one. In the case of the loop given above, a fourth instruction has to be added, slightly slowing down the responsiveness of detecting an idle device.

Memory-Mapped I/O...

Disadvantages:

- Adds extra complexity to both the hardware and the OS because caching of device control registers have to be disabled. We know that most modern computers have some form of caching of memory words, but caching of device control registers could be disastrous. For example, suppose the assembly code loop given above in the presence of caching. The first reference to `PORT_4` would cause it to be cached. Subsequent references would just take the value from the cache and not even ask the device. Then when the device finally became ready, the software would have no way of finding it out. Instead, the loop would go on forever.
- If there is only one address space, then all memory modules and all I/O devices must examine all memory references to see which one to respond to.

Direct Memory Access

- No matter whether a CPU does or does not have memory-mapped I/O, it needs to address the device controllers to exchange data with them. The CPU can request data from an I/O controller one byte at a time but doing so *wastes the CPU's time*, so a different scheme, called **DMA (Direct Memory Access)** is used.
- The OS can only use DMA if the hardware has a **DMA controller**. Many computers avoid burdening the CPU by offloading some of its work to a special propose processor called **DMA controller**. More commonly, a single DMA controller is available (in-built in parentboard) for regulating transfers to multiple devices but some systems may have it integrated with disk controllers.
- The DMA controller has access to the system bus independent of the CPU as shown in figure below. It contains several registers that can be written and read by the CPU. These include a **memory address register**, a **byte count register**, and **one or more control registers**. The control registers specify the I/O port to use, the direction of the transfer (reading from the I/O device or writing to the I/O device), the transfer unit (byte at a time or word at a time), and the number of bytes to transfer in one burst.

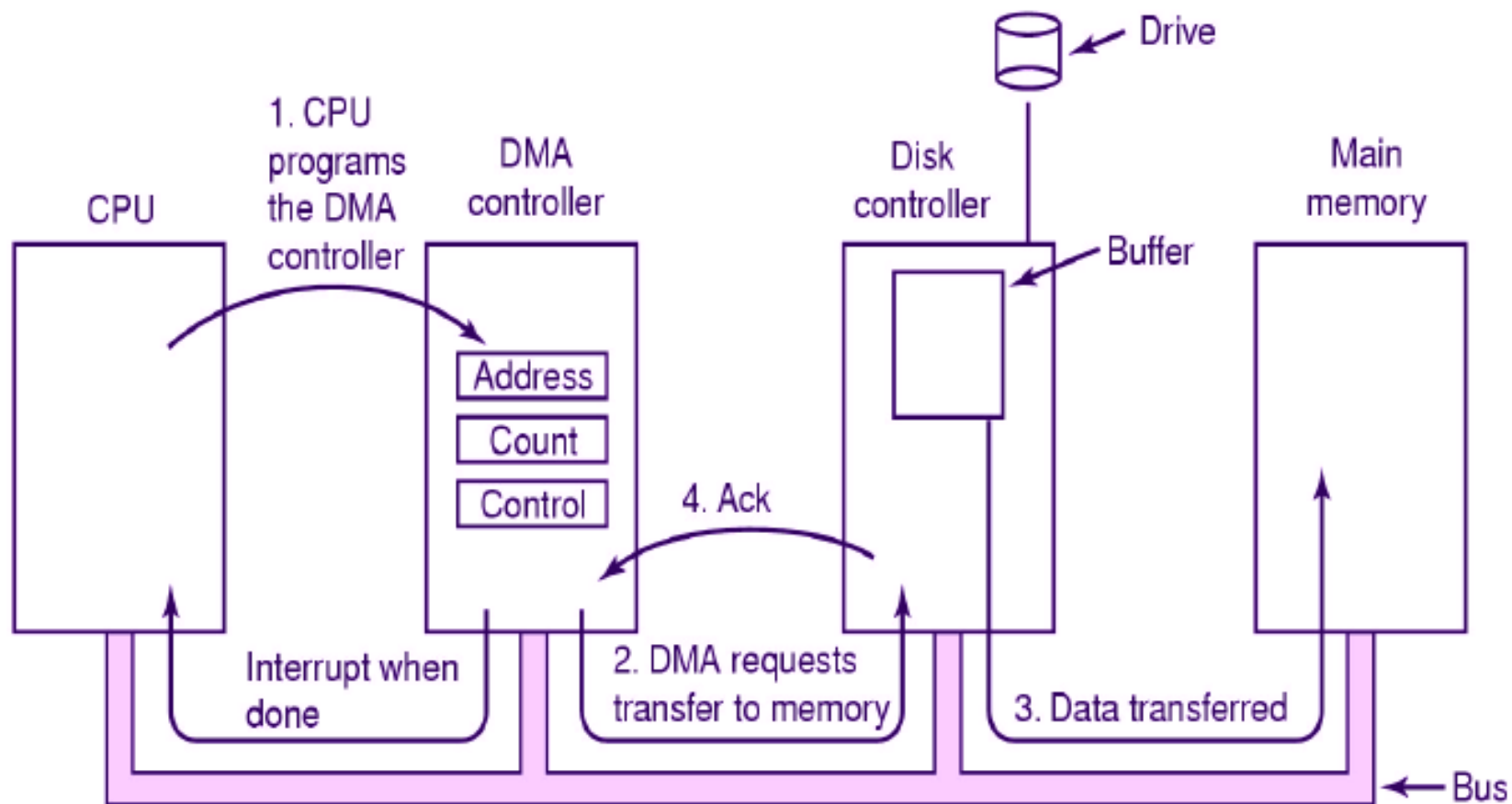


Figure: Operation of a DMA transfer

Direct Memory Access...

How DMA works?

1. The CPU programs the DMA controller by setting its registers so it knows what to transfer where. It also issues a command to the disk controller telling it to read data from disk to its internal buffer and verify the checksum. When valid data are in disk controller's buffer, DMA can begin.
2. The DMA controller initiates the transfer by issuing a read request over the bus to the disk controller. The memory address to write to is on the bus' address lines so when the disk controller fetches the next word from its internal buffer, it knows where to write to.
3. Data is transferred from disk controller to memory.
4. When the write is complete, the disk controller sends an acknowledgment signal to DMA controller, also over the bus. The DMA controller then increments the memory address to use and decrements the byte count. This continues until the byte count is not equal to 0.
5. When transfer is completed, the DMA controller interrupt the CPU. Now when the OS starts up, it does not have to copy the disk block to memory as it is already there.

Reading Assignment

- Textbook: Tanenbaum (Topic: DMA) Pages: 336-339
- Reference: Silberschatz (Topic: DMA) Pages: 464-465

How does DMA increase system concurrency (merit)?

How does it complicate the hardware design (demerit)?

[**Hint:** Not all computers use DMA. The argument against it is that the main CPU is often far faster than the DMA controller and can do the job much faster (when the limiting factor is not the speed of the I/O device). If there is no other work for it to do, having the (fast) CPU wait for the (slow) DMA controller to finish is pointless. Also, getting rid of the DMA controller and having the CPU do all the work in software saves money, important on low-end (embedded) computers.]

I/O Handling

Polling:

- Processing is interrupted at brief intervals to allow the CPU to check back to I/O to see if the I/O operation has completed (**BUSY WAITING**).
- Well mannered way of getting attention.
- Advantage: Simple
- Problem: May bear long wait and can be a high overhead operation becoming inefficient.
- *Used in embedded system, where CPU has nothing else to do.*

I/O Handling...

Interrupt:

- The hardware mechanism that enables a device to notify the CPU is called an ***interrupt***. Interrupt forces the CPU to stop what it is doing and start doing something else.
- **Advantage:** Improves efficiency.
- **Problem:** Because of the selfish nature, the first interrupt may not be served if the similar second happened before the time needed to serve the first.

I/O Handling...

Example:

Scenario: A cook is cooking something in modern microwave oven equipped kitchen.

CASE A: The cook may regularly peek through the oven's glass door and watch as something is cooked in it; this kind of regular monitoring is ***polling***.

CASE B: The cook may set a timer to expire after an appropriate number of minutes; the *buzzer* sounding after this interval is an ***interrupt***.

TU Exam Question 2068

- Which one suited, polling/interrupt, for the following types of system? Give reason.
 - (a) A system dedicated to controlling single I/O devices.
 - (b) A workstation running as heavily used web server.
- **Answer:** (a) **A system dedicated to controlling single I/O devices:** Polling is suited for the system dedicated to controlling single I/O device. The host repeatedly checks the busy bit on the device until it becomes clear. Polling can be very fast and efficient, if both the device and the controller are fast and if there is significant data to transfer. It becomes inefficient, however, if the host must wait a long time in the busy loop waiting for the device, or if frequent checks need to be made for data that is infrequently there.
- (b) **A workstation running as heavily used web server:** Interrupt is suited for workstation running as heavily used web server. Interrupts allow devices to notify the CPU when they have data to transfer or when an operation is complete, allowing the CPU to perform other duties when no I/O transfers need its immediate attention. The CPU has an interrupt-request line that is sensed after every instruction. The interrupt handler determines the cause of the interrupt, performs the necessary processing, performs a state restore, and executes a return from interrupt instruction to return control to the CPU.

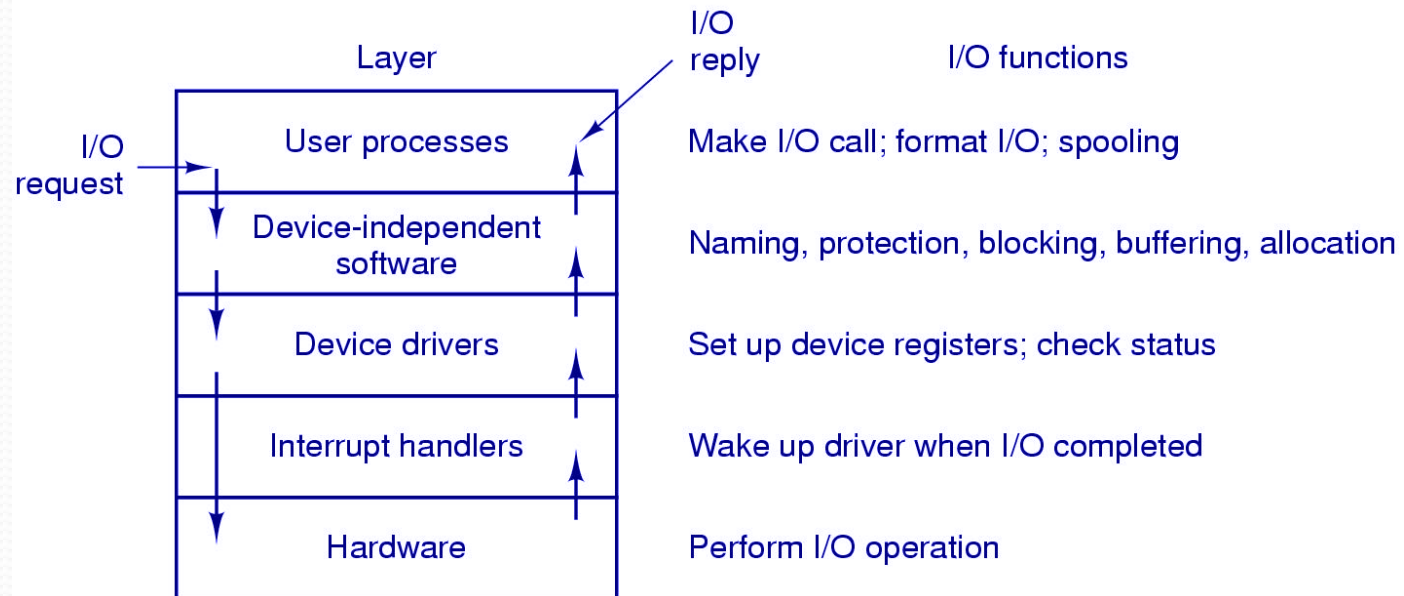
Principles of I/O Software

Goals of the I/O Software:

- **Device Independence:** It should be possible to write programs that can access any I/O device without having to specify the device in advance. For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.
- **Uniform Naming:** The name of a file or a device should simply be a string or an integer and not depend on the device in any way.
- **Error Handling:** Errors should be handled as close to the hardware as possible. If the controller discovers a read error, it should try to correct the error itself if it can. If it cannot, then the device driver should handle it, perhaps by just trying to read the block again. Many errors are transient, such as read errors caused by specks of dust on the read head, and will go away if the operation is repeated. Only if the lower layers are not able to deal with the problem should the upper layers be told about it. In many cases, error recovery can be done transparently at a low level without the upper levels even knowing about the error.
- **Synchronous (blocking) versus Asynchronous (interrupt-driven) transfer:** Most physical I/O is asynchronous—the CPU starts the transfer and goes off to do something else until the interrupt arrives. User programs are much easier to write if the I/O operations are blocking—after a read system call the program is automatically suspended until the data are available in the buffer. It is up to the operating system to make operations that are actually interrupt-driven look blocking to the user programs.
- **Buffering:** Often data that come off a device cannot be stored directly in its final destination. For example, when a packet comes in off the network, the operating system does not know where to put it until it has stored the packet somewhere and examined it. Also, some devices have severe real-time constraints (for example, digital audio devices), so the data must be put into an output buffer in advance to decouple the rate at which the buffer is filled from the rate at which it is emptied, in order to avoid buffer underruns. Buffering involves considerable copying and often has a major impact on I/O performance.

I/O Software Layers

- I/O software is organized into many layers, as shown in figure below. Each layer has a well-defined function to perform and a well-defined interface to the adjacent layers. The functionality and interfaces differ from system to system.



I/O Software Layers...

- The arrows in the figure shows the flow of control.
- When a user process tries to read a block from a file, for example, the operating system is invoked to carry out the call. The device-independent software looks for it in the buffer cache, for example. If the needed block is not there, it calls the device driver to issue the request to the hardware to go get it from the disk. The process is then blocked until the disk operation has completed.
- When the disk is finished, the hardware generates an interrupt. The interrupt handler is run to discover what has happened, that is, which device wants attention right now. It then extracts the status from the device and wakes up the sleeping process to finish off the I/O request and let the user process continue.

Interrupt Handlers

- Block the device driver until the I/O operation has completed and the interrupt occurs.
- The interrupt-handler determines the cause of the interrupt and performs the necessary processing. The various steps in interrupt handling are:
 1. Save registers.
 2. Set up a stack for the interrupt service procedure.
 3. Acknowledge the interrupt controller.
 4. Copy registers to the process table.
 5. Run the interrupt service procedure.
 6. Set up the MMU context for the process to run next.
 7. Load new process' registers.
 8. Start running the new process.

Device Drivers

- Each I/O device attached to a computer needs some device-specific code for controlling it. This code is called ***device driver***, and is generally written by the device's manufacturer and delivered along with the device.
- Each device driver normally handles one device type, or at most, one class of closely related devices. For example, a SCSI disk driver can usually handle multiple SCSI disks of different sizes and different speeds, and perhaps a SCSI CD-ROM as well. On the other hand, a mouse and joystick are so different that different drivers are usually required.
- **Functions:**
 - Accept read and write requests from the device independent software above it.
 - Initialize the devices, if necessary.
 - Manage power requirement and log events.
 - It checks the status of devices - in use or free.
 - Decides which command to issue to the device controller's device registers if there is command queue.

Device-Independent OS Software

- The basic function of the device independent-software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level-software.
- Some common functions in the device-independent software are:
 1. Uniform interfacing for device drivers - naming and protection.
 2. Buffering.
 3. Error reporting.
 4. Allocating and releasing dedicated devices.
 5. Providing a device-independent block size.

User Processes

- System calls, including the I/O system calls, are normally made by the user processes via library procedures.
- When a C program contains the call
count=write(fd, buffer, nbytes);
the library procedure ***write*** will be linked with the program and contained in the binary program present in memory at run time. The collection of all these procedures is clearly part of the I/O system.
- While, these procedures just put their parameters in the appropriate place for the system call, there are other procedures that do the real work. In particular, formatting of input and output is done by library procedures like *printf()* and *scanf()*.
- ***Spooling***: It is a way of dealing with dedicated I/O devices in a multiprogramming system. Consider a typical spooled device: a printer. Although it would be technically easy to let any user process open the character special file for the printer, suppose a process opened it and then did nothing for hours. No other process could print anything. So, instead what is done is to create a special process, called a ***daemon***, and a special directory, called a ***spooling directory***. To print a file, a process first generates the entire file to be printed and puts it in the spooling directory. It is up to the daemon, which is the only process having permission to use the printer's special file, to print the files in the directory. By protecting the special file against direct use by users, the problem of having someone keeping it open unnecessarily long is eliminated.

TU Model Question

- a) How is the Direct Memory Access (DMA) set up?
- b) Explain the concept of Buffering.
- c) How interrupt is enabling and detected?

TU Exam Question 2066

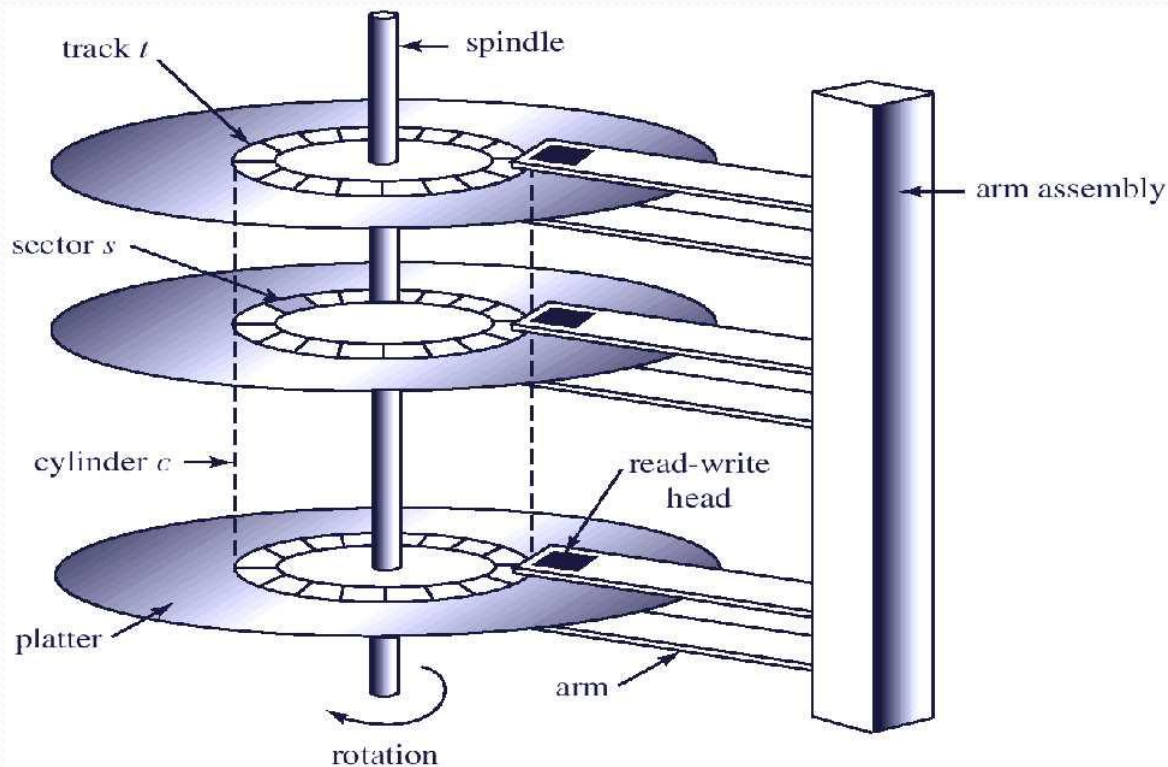
- What do you mean by interrupt? Explain the working mechanism of interrupt controller.

Disk Management

Disks are I/O devices common to every computer. The most common ones are magnetic disks (hard disks and floppy disks). They are characterized by the fact that reads and writes are equally fast, which makes them ideal as secondary memory (paging, file systems, etc.). Arrays of these disks are sometimes used to provide highly reliable storage. For distribution of programs, data and movies, various kinds of optical disks (CD-ROMs, CD-Recordables, CD-Rewritables and DVDs) are also important.

Disk Structure

- Disk surface is divided into a number of logical blocks called *sectors* and *tracks*. The term *cylinder* refers to all the tracks at particular head position in hard disk.



Disk Operations

- ***Seek***: It is the process of moving the arm assembly to the new cylinder (which contains the desired sector). So the time to move the arm to the proper cylinder is called ***seek time***.
- ***Rotational Latency*** (or ***Rotational Delay***): It is the additional time taken to rotate from its current position to a position adjacent to the read/write head.
- To access a particular record, first the arm assembly must be moved to the appropriate cylinder, and then rotate the disk until it is immediately under the read/write head.
- The time taken to access the whole record is called ***transmission time*** (or ***actual data transfer time***).

Disk Scheduling

- OS is responsible to use the hardware efficiently - for the disk drive this means fast seek, latency and transmission time.
- For most disks, the seek time dominates the other two times, so reducing the mean seek time can improve system performance substantially.

Disk Scheduling Algorithms

- *FCFS (First-Come, First Served) Scheduling*
- *SSTF (Shortest Seek Time First) Scheduling*
- *SCAN Scheduling (Elevator Algorithm)*
- *C-SCAN Scheduling (Circular SCAN algorithm)*
- *LOOK Scheduling*
- *C-LOOK Scheduling*

FCFS Scheduling

- The first request to arrive is the first one to be serviced.
- For example, consider a disk queue with requests for I/O to blocks on cylinders

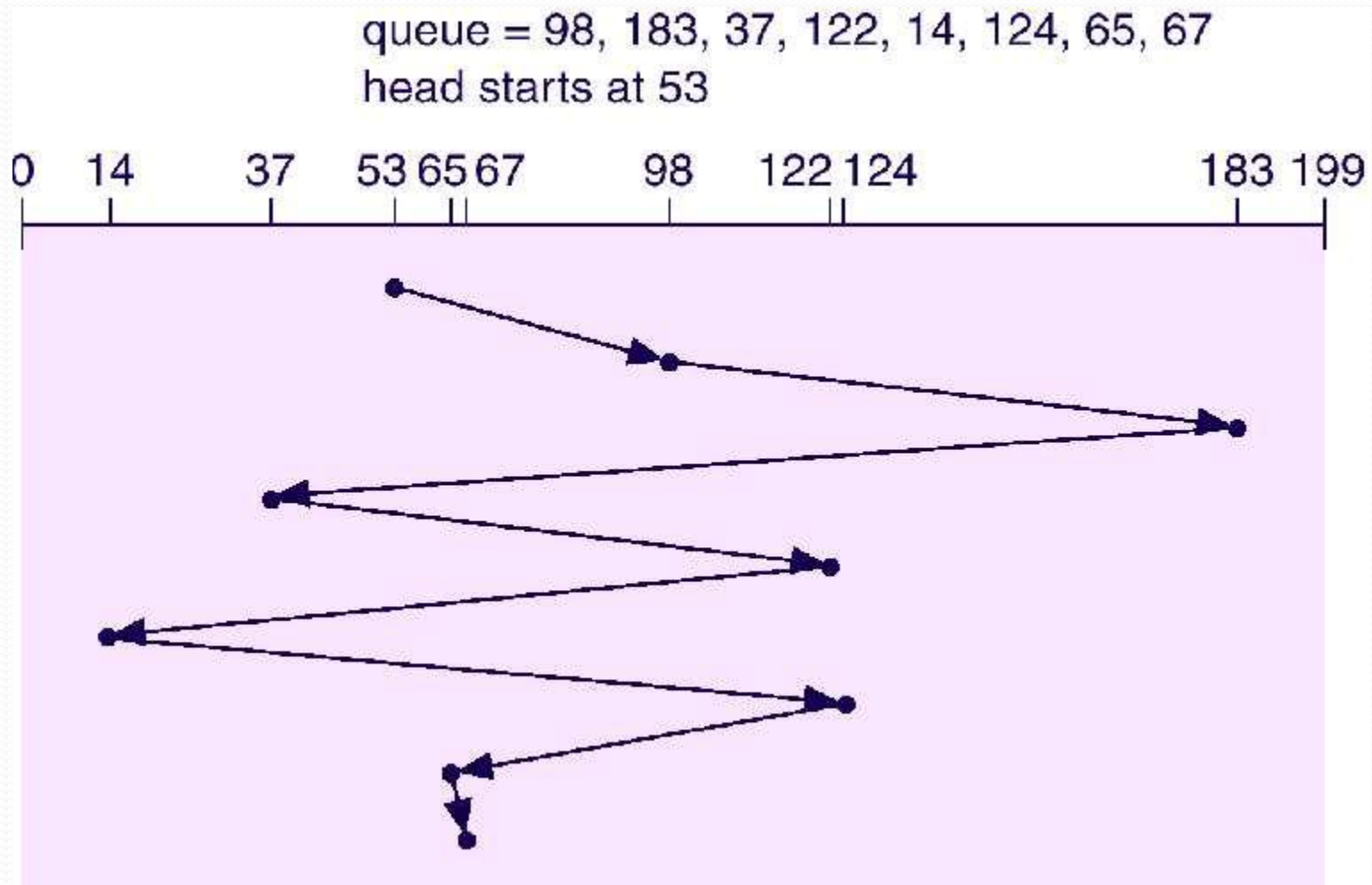
98, 183, 37, 122, 14, 124, 65, 67

in that order. If the disk head is initially at cylinder 53, it will first move from 53 to 98, then to 183, 37, 122, 14, 124, 65, and finally to 67, for a total head movement of $((98-53)+(183-98)+(183-37)+(122-37)+(122-14)+(124-14)+(124-65)+(67-65))$ 640 cylinders.

Advantage: Simple and fair.

Problem: Does not provide fast service.

FCFS Scheduling...

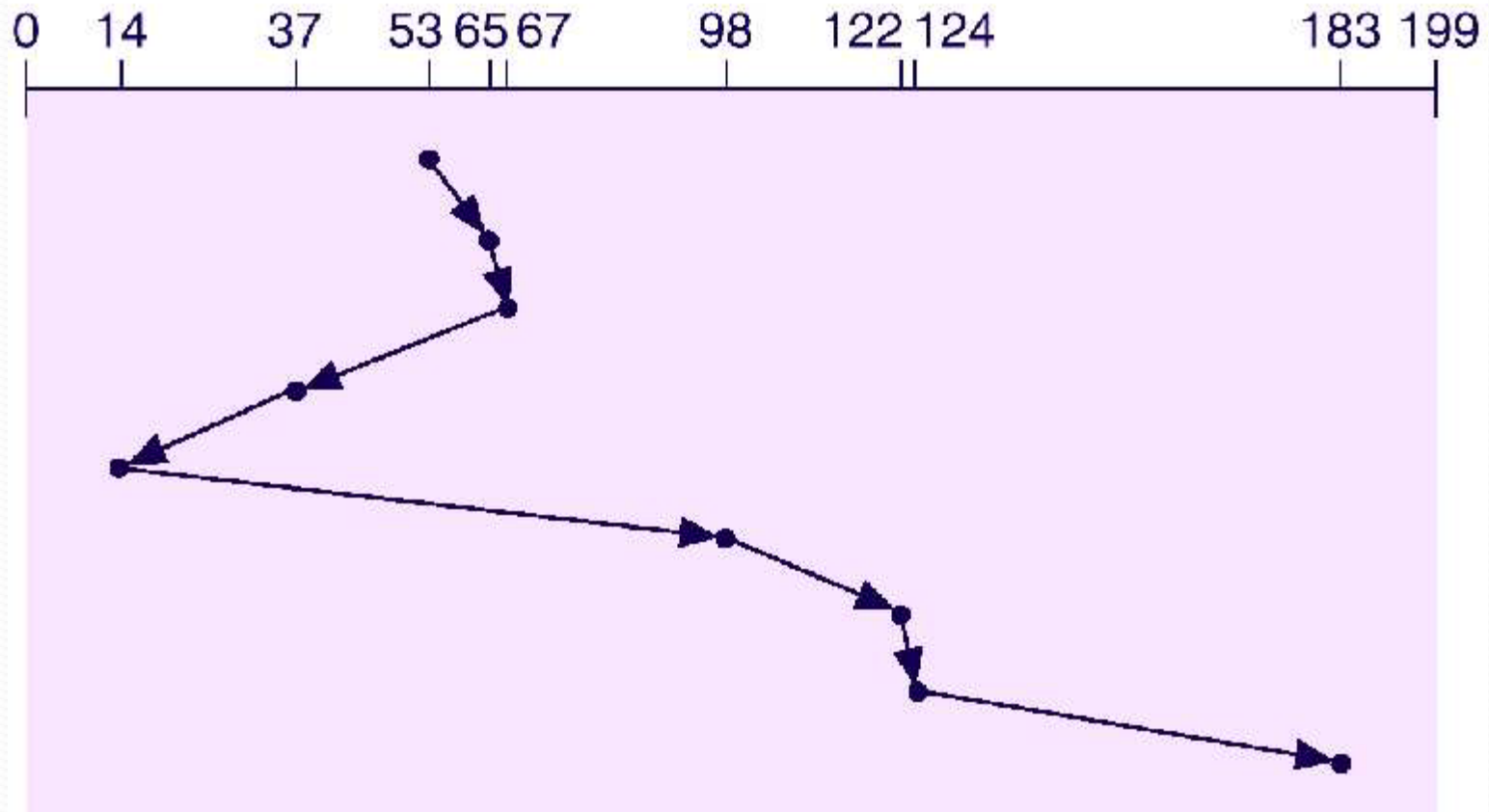


SSTF Scheduling

- It selects the request with the minimum seek time from the current head position. Since seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position.
- For our example request queue, the closest request to the initial head position (53) is at cylinder 65. Once we are at cylinder 65, the next closest request is at cylinder 67. From there, the request at cylinder 37 is closer than 98, so 37 is served next. Continuing, we service the request at cylinder 14, then 98, 122, 124 and finally 183. This scheduling method results in a total head movement of only 236 cylinders.

SSTF Scheduling...

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



SSTF Scheduling...

- **Advantage:** This algorithm gives a substantial improvement in performance over FCFS scheduling.
- **Problem:** SSTF scheduling is a form of SJF scheduling, and, like SJF scheduling, it may cause **starvation** of some requests. Note that requests may arrive at any time. Suppose that we have two requests in the queue, for cylinders 14 and 186, and while servicing the request from 14, a new request near 14 arrives. This new request will be serviced next, making the request at 186 wait. While this request is being serviced, another request close to 14 could arrive. In theory, a continual stream of requests near one another could arrive, causing the request for cylinder 186 to wait indefinitely. Thus it is not an optimal solution.
- *It is used in batch system where throughput is the major consideration but unacceptable in an interactive system.*

SCAN Scheduling

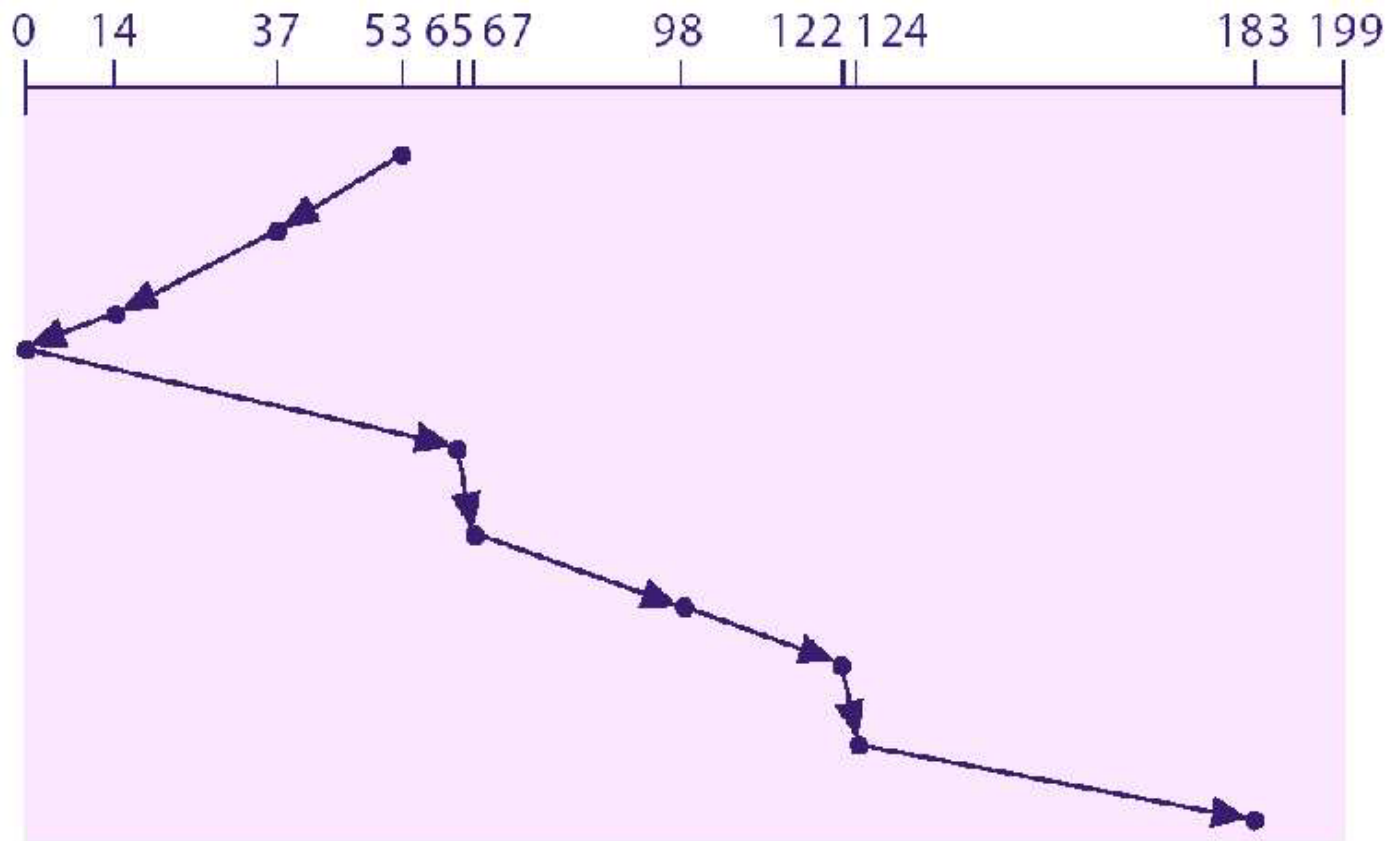
- In the **SCAN algorithm**, the disk arm starts at one end of the disk, and moves toward the other end, servicing requests as it reaches each cylinder, until it gets to the other end of the disk. At the other end, the direction of head movement is reversed, and servicing continues. The head continuously scans back and forth across the disk.
- Sometimes called the *elevator algorithm* because this algorithm works just like elevators, which keep moving in the same direction until there are no more requests in that direction and then switching the directions.
- **Advantage:** Decreases variances in seek and improves response time.
- **Problem:** Starvation is possible if there are repeated requests in current track.

SCAN Scheduling...

- Let us consider our example again.
- Before applying SCAN to schedule the requests on cylinders 98, 183, 37, 122, 14, 124, 65 and 67, we need to know the direction of head movement, in addition to the head's current position (53). If the disk arm is moving toward 0, the head will service 37 and then 14. At cylinder 0, the arm will reverse and will move toward the other end of the disk, servicing the requests at 65, 67, 98, 122, 124, and 183. If a request arrives in the queue just in front of the head, it will be serviced almost immediately; a request arriving just behind the head will have to wait until the arm moves to the end of the disk, reverses direction and comes back.

SCAN Scheduling...

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



**Total head
movements
= 236**

TU Exam Question 2066

- Compare the throughput (overall performance) of SCAN with SSTF.

TU Exam Question 2066

- Suppose that a disk drive has 100 cylinders, numbered 0 to 99. The drive is currently serving a request at cylinder 43, and previous request was at cylinder 25. The queue of pending request, in FIFO order is:

86, 70, 13, 74, 48, 9, 22, 50, 30

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all pending requests for each of the following disk scheduling algorithms?

- a. FCFS
- b. SCAN

Answer:

- (a) FCFS: Total distance (in cylinders) that the disk arm moves to satisfy all pending request
$$= (86-43) + (86-70) + (70-13) + (74-13) + (74-48) + (48-9) + (22-9) + (50-22) + (50-30)$$
$$= 43+16+57+61+26+39+13+28+20$$
$$= 303$$

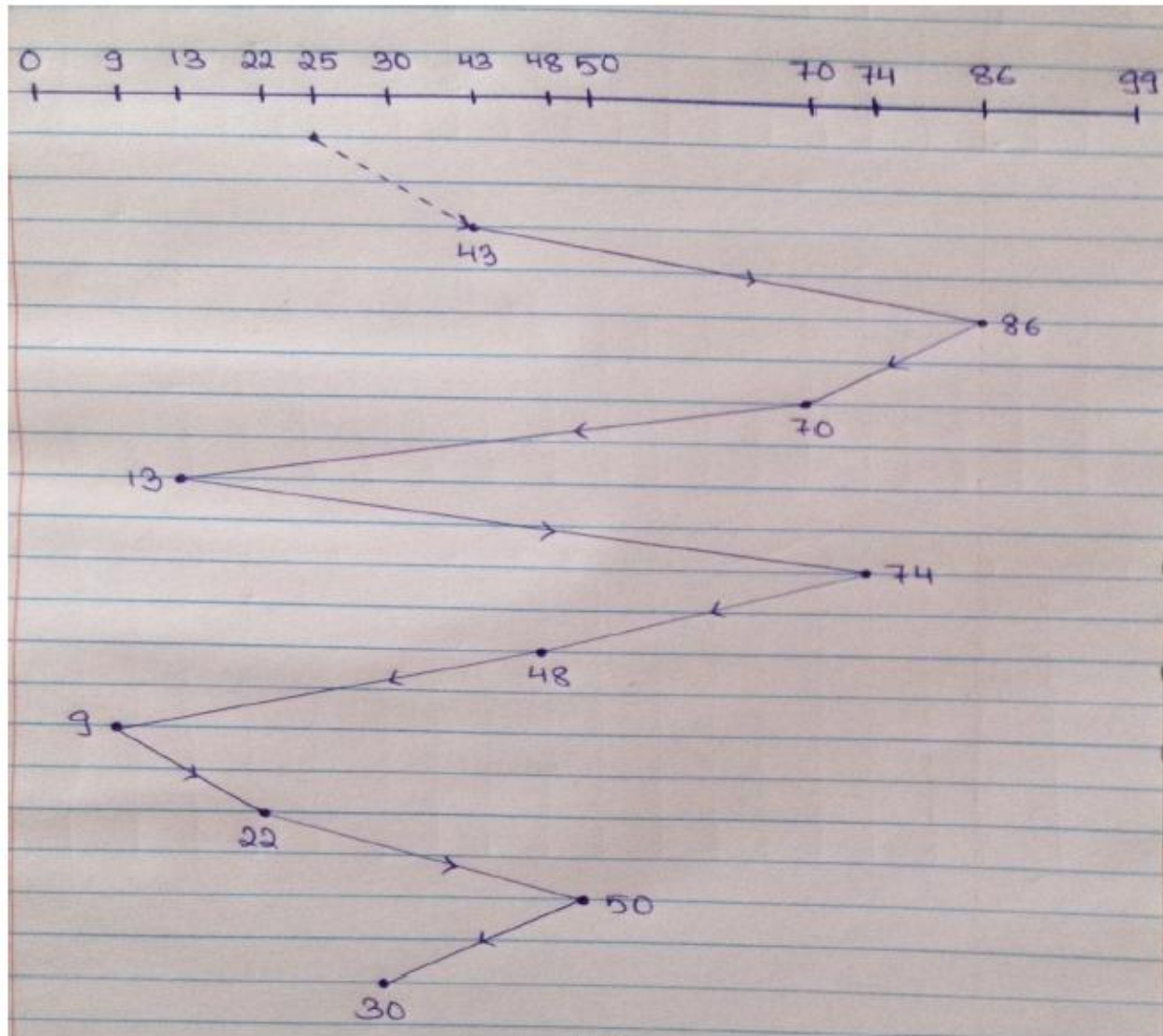


Fig: FCFS

Answer:

- **(b) SCAN:** Total distance (in cylinders) that the disk arm moves to satisfy all pending request
$$= (48-43) + (50-48) + (70-50) + (74-70) + (86-74) + (99-86) + (99-30) + (30-22) + (22-13) + (13-9)$$
$$= 5+2+20+4+12+13+69+8+9+4$$
$$= 146.$$

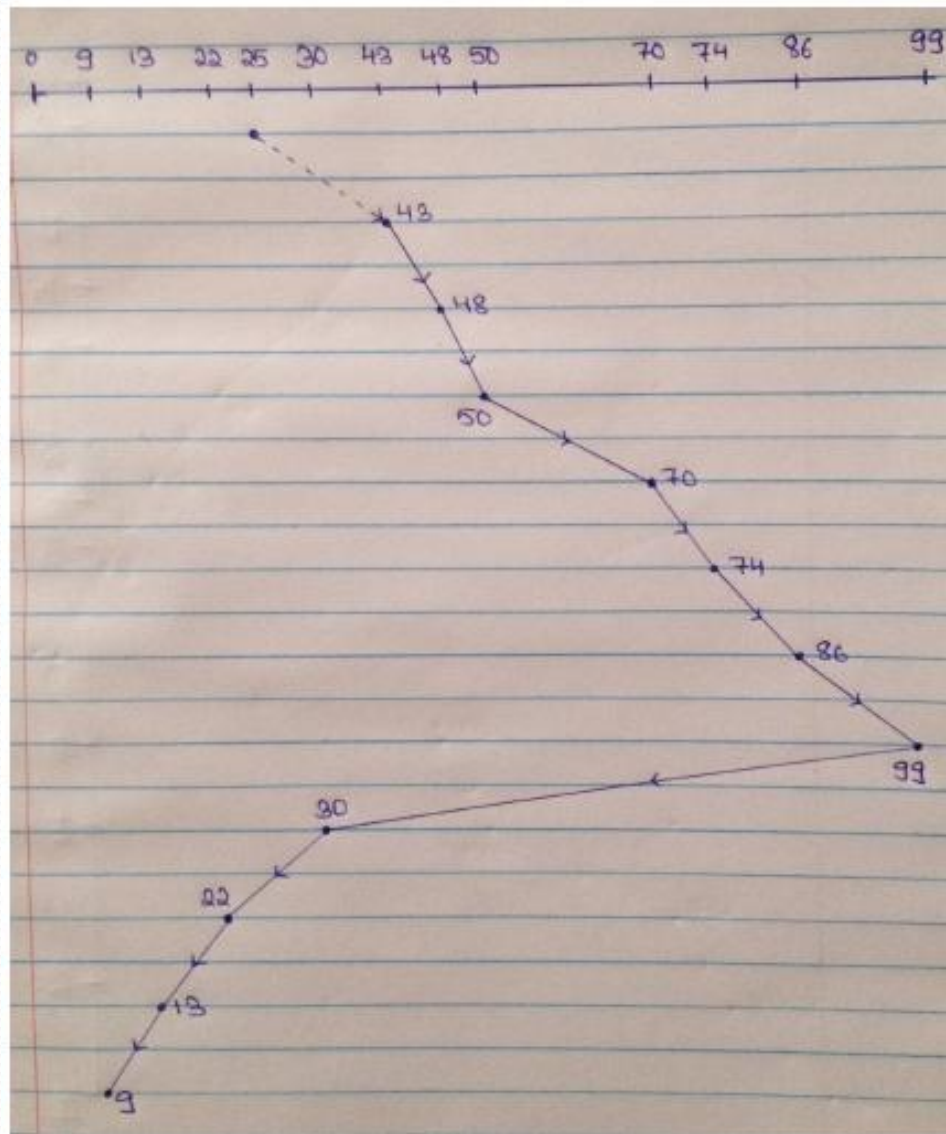


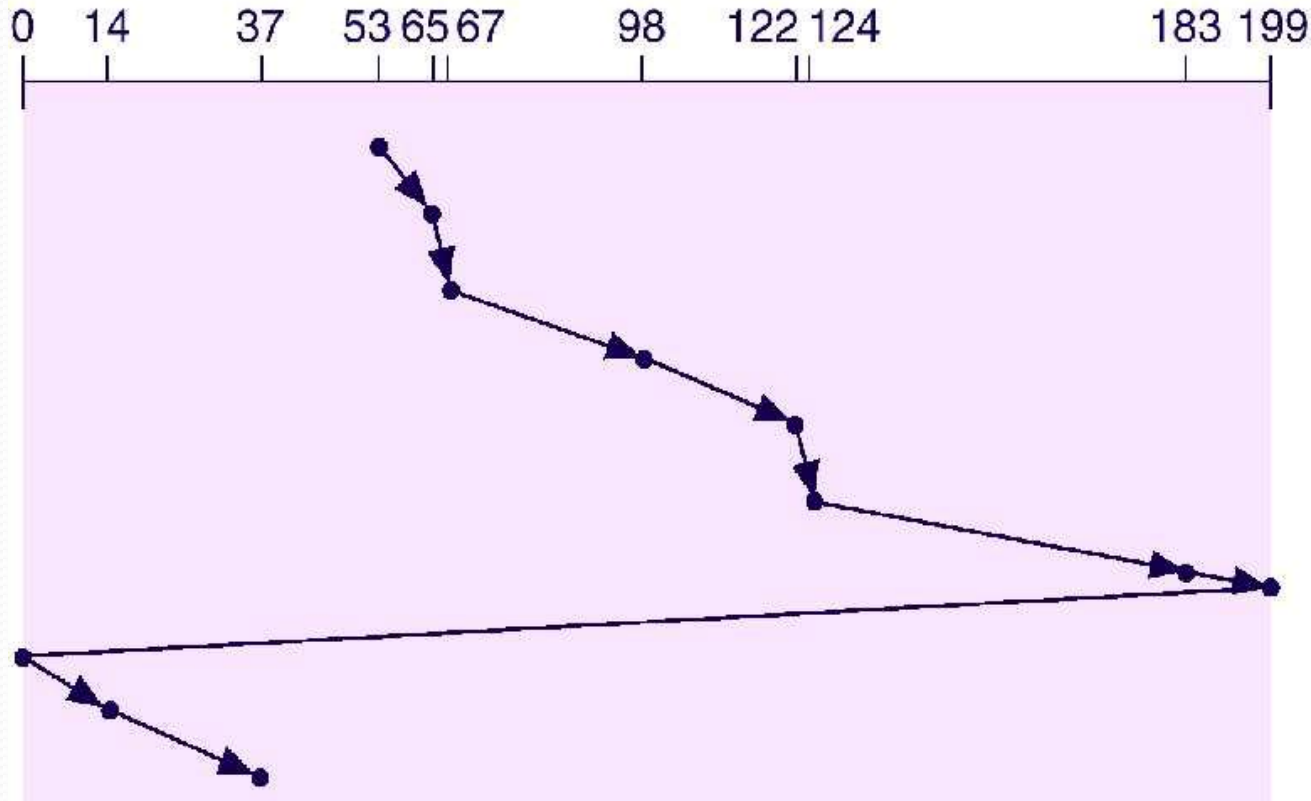
Fig: SCAN

C-SCAN Scheduling

- When a uniform distribution of requests for cylinders is present, if we consider the density of requests when the head reaches one end and reverses direction, we will find that there are relatively few requests immediately in front of the head, since these cylinders have recently been serviced. The heaviest density of requests is at the other end of the disk. These requests have also waited, so why not go there first?
- Circular SCAN (C-SCAN) scheduling is a variant of SCAN designed to provide a more uniform wait time.
- Like SCAN, C-SCAN moves the head from one end of the disk to the other, servicing requests along the way. When the head reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- The C-SCAN scheduling algorithm treats the cylinders as a circular list that wraps around from the final cylinder to the first one.

C-SCAN Scheduling...

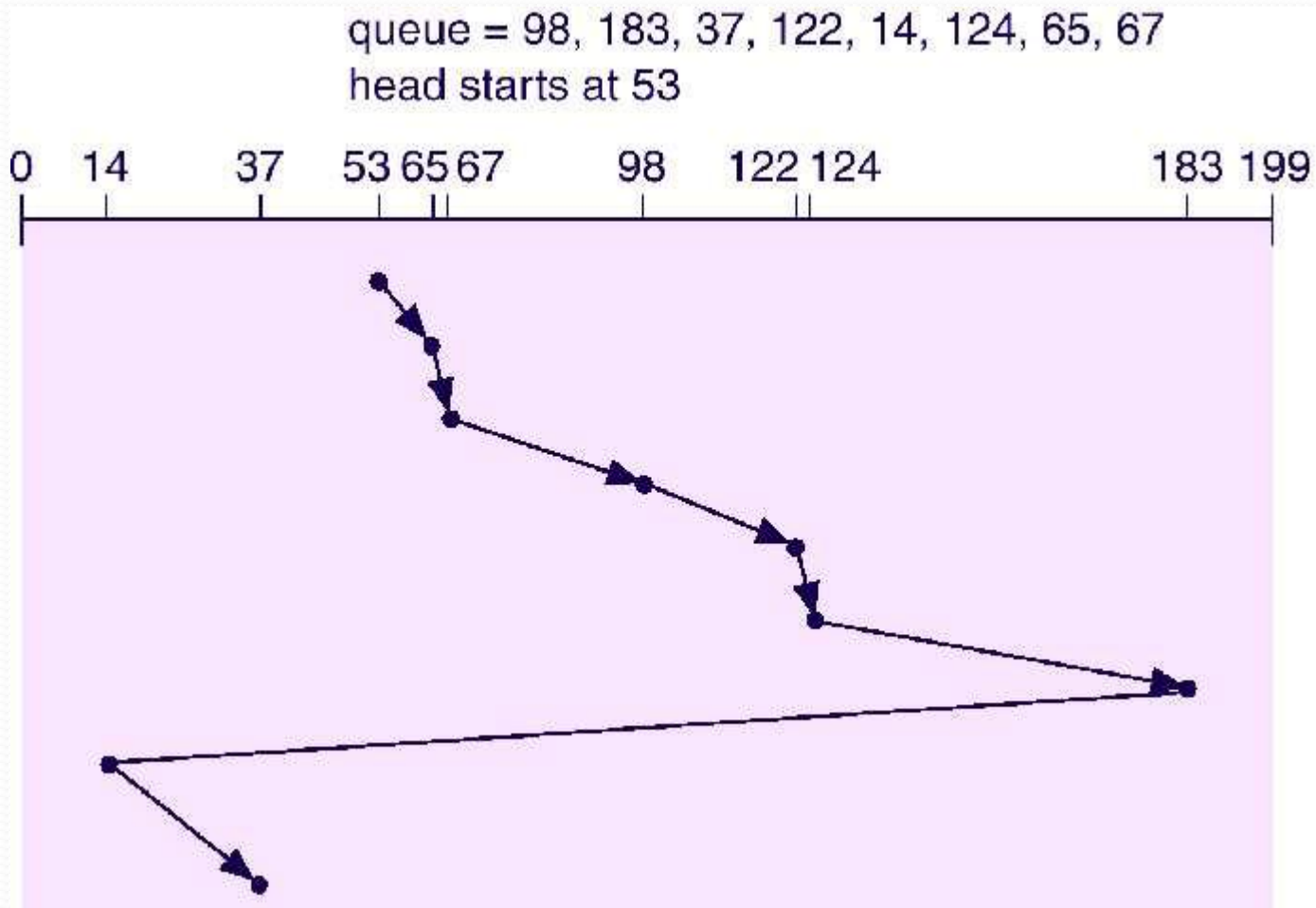
queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



LOOK and C-LOOK Scheduling

- LOOK and C-LOOK are practical versions of SCAN and C-SCAN scheduling algorithms respectively, in which the disk arm does not move across the full width of the disk. Rather the arm goes only as far as the final request in each direction. Then, it reverses direction immediately, without going all the way to the end of the disk.
- These are called LOOK and C-LOOK scheduling because they *look* for a request before continuing to move in a given direction.

C-LOOK Scheduling...



Classwork

Suppose that a disk drive has 5,000 cylinders, numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of the pending requests, in FIFO order, is

86, 1470, 913, 1774, 948, 1502, 1022, 1750, 130.

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms?

a) FCFS b) SSTF c) SCAN d) C-SCAN (e) LOOK f) C-LOOK

Answer: (Checked and Found Correct)

- a) The FCFS schedule is 143, 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130. The total seek distance is 7081.
- b) The SSTF schedule is 143, 130, 86, 913, 948, 1022, 1470, 1509, 1750, 1774. The total seek distance is 1745.
- c) The SCAN schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 130, 86. The total seek distance is 9769.
- d) The C-SCAN schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 4999, 0, 86, 130. The total seek distance is 9985. (According to Silberschatz book: 9813).
- e) The LOOK schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 130, 86. The total seek distance is 3319.
- f) The C-LOOK schedule is 143, 913, 948, 1022, 1470, 1509, 1750, 1774, 86, 130. The total seek distance is 3363.

Background: Disk Reliability

- Disk drives have continued to get smaller and cheaper, so it is now economically feasible to attach a large number of disks to a computer system.
- However, the chance that some disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail. For example, suppose that the *mean time to failure* of a single disk is 100,000 hours. Then, the mean time to failure of some disk in an array of 100 disks will be $100,000/100 = 1,000$ hours, or 41.66 days, which is not long at all. If we store only one copy of the data, then each disk failure will result in loss of a significant amount of data – such a high rate of data loss is unacceptable.
- The solution to the problem of reliability is to introduce *redundancy*; we store extra information that is not needed normally, but that can be used in the event of failure of a disk to rebuild the lost information. Thus, even if a disk fails, data are not lost.
- The simplest approach to introduce redundancy is to duplicate every disk. This technique is called *mirroring* (or *shadowing*). A logical disk then contains two physical disks, and every write is carried out on both disks.

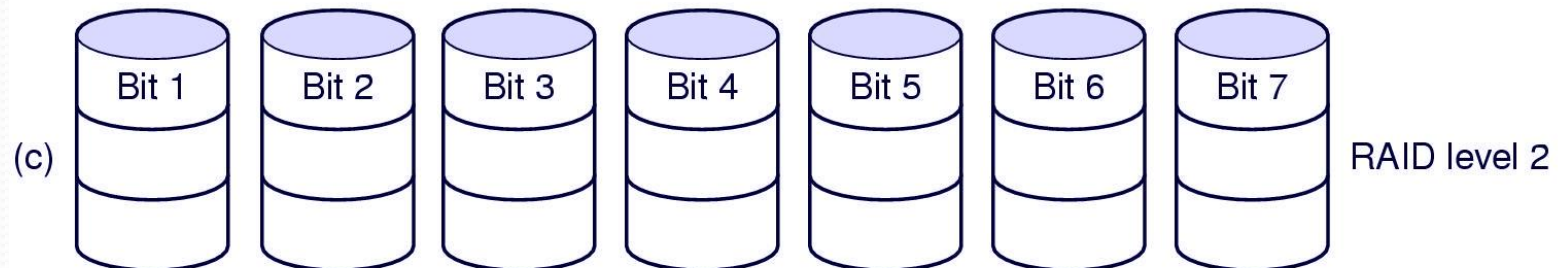
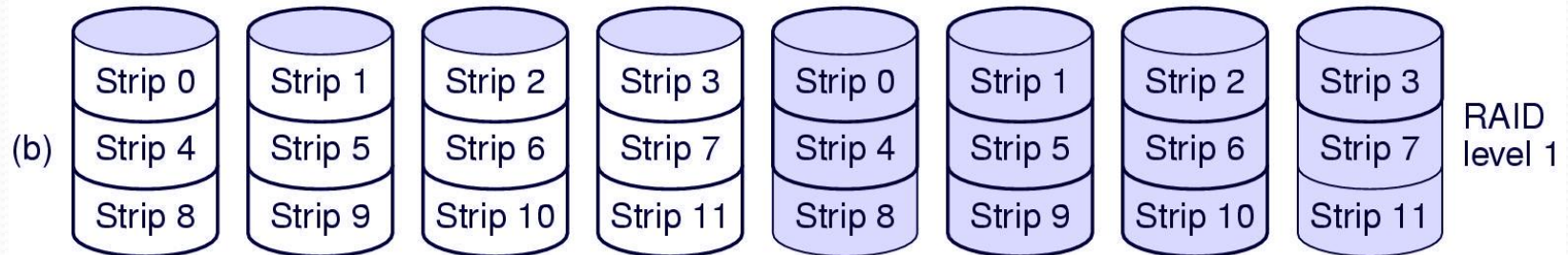
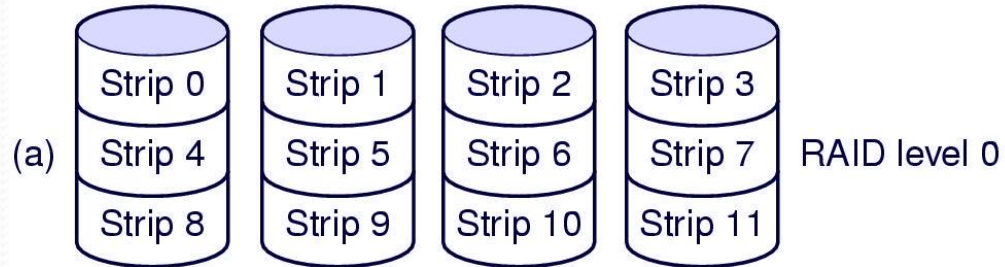
Background: Disk Performance

- With disk mirroring, the rate at which read requests can be handled is doubled, since read requests can be sent to either disk and is done in parallel. The transfer rate of each read is the same as in a single disk system, however the number of reads per unit time has doubled.
- With multiple disks, the transfer rate can also be improved by **striping** data across multiple disks. In this method, the bits of each byte are split across multiple disks called **bit-level striping**. For example, if we have an array of eight disks, we write bit i of each byte to disk i . The array of eight disks are treated as a single disk with sectors that are eight times the normal size, and having eight times the access rate. In such an organization, every disk participates in every access (read or write), so the number of accesses that can be processed per second is about the same as on a single disk, but each access can read eight times as many data in the same time as on a single disk.
- Bit-level striping can be generalized to a number of disks that either is a multiple of 8 or divides 8. For example, if we use an array of four disks, bits i and $4+i$ of each byte go to disk i . Further, striping does not need to be at the level of bits of a byte: For example, in **block-level striping**, blocks of a file are striped across multiple disks; with n disks, block i of a file goes to disk $(i \bmod n) + 1$. Other levels of striping, such as bytes of a sector or sectors of a block are also possible.

RAID

- It stands for ***Redundant Array of Inexpensive (Independent) Disks***.
- There is a huge gap between CPU performance and disk performance. The CPU's performance increases exponentially roughly doubling every 18 months (Moore, CEO Intel). However disk seek time (currently slightly under 10 msec) increases slowly over time. A way is needed to improve the disk performance and also their reliability. For this Patterson et al. suggested using *RAID*.
- In RAID, multiple disks are organized in a way that improves the disk performance and their reliability.
- RAID allows more than one disk to be used for a given operation, and allows continued operation and even automatic recovery in the face of disk failure.
- Implemented using a RAID controller that looks like a SLED (***Single Large Expensive Disk***) to the OS.
- There are six different organizations possible using RAIDs called RAID levels from RAID level 0 to RAID level 5. All RAIDs have the property that data are distributed over the drives, to allow parallel operation.

RAID levels



RAID level 0

- RAID level 0 creates one large virtual disk from a number of smaller disks.
- Storage is grouped into logical units called strips with the size of a strip being some multiple of sector size called blocks.
- The virtual storage is sequence of strips interleaved among the disks in the array.
- No redundancy (such as mirroring).
- **Advantage:** Can create large disk and performance benefit can be achieved.
- **Disadvantage:** Reliability decreases.

RAID level 1

- RAID level 1 stores duplicate copy of each strip, with each copy on a different disk (***mirroring***). Thus there are four primary disks and four backup disks.
- **Advantage:**
 - Excellent reliability; if drive crashes, the copy is used.
 - Twice the read performance can be achieved by distributing the load over more drives.
- **Disadvantage:** On a write, every strip is written twice. Write Performance is no better than in a single drive.

RAID level 2

- RAID level 2 splits each byte of the single virtual disk into a pair of 4-bit nibbles and then adds a Hamming(7,4) code to each one to form a 7-bit word, of which bits 1, 2 and 4 are parity bits required for *error correction*. Thus the 7-bit Hamming coded word is written over the seven drives, one bit per drive.
- **Advantage:** Total parallelism.
- **Disadvantages**
 - Requires more number of drives for error correction.
 - All drives need to be rotationally synchronized.
 - Hamming checksum is needed every bit time.

What is Hamming Code?

- In coding theory, Hamming(7,4) is an error-correcting code that encodes 4 bits of data into 7 bits by adding 3 parity bits. It is a member of a large family of Hamming codes that were invented by Richard Hamming.
- Hamming's (7,4) algorithm can correct any single bit error, or detect all single-bit and two-bit errors.
- The key to Hamming code is the use of extra parity bits to allow the identification of a single error.

Creating a Hamming Code Word

- Mark all bit positions that are powers of two as parity bits (positions 1, 2, 4, 8, 16, 32, 64, etc).
- All other positions are for the data to be encoded (positions 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 17, etc).
- Each parity bit calculates the parity for some of the bits in the code word. The position of the parity bit determines the sequence of bits that it alternately checks and skips.
 - Position 1: Check 1 bit, skip 1 bit (1, 3, 5, 7, 9, 11, 13 etc).
 - Position 2: Check 2 bits, skip 2 bits (2, 3, 6, 7, 10, 11, 14, 15 etc).
 - Position 4: Check 4 bits, skip 4 bits (4, 5, 6, 7, 12, 13, 14, 15 etc).
 - Position 8: Check 8 bits, skip 8 bits (8-15, 24-31, etc).
 - Position 16: Check 16 bits, skip 16 bits (16-31, 48-63, etc).
- Set a parity bit to 1 if the total number of 1's in the positions it checks is odd. Otherwise set the parity to 0.

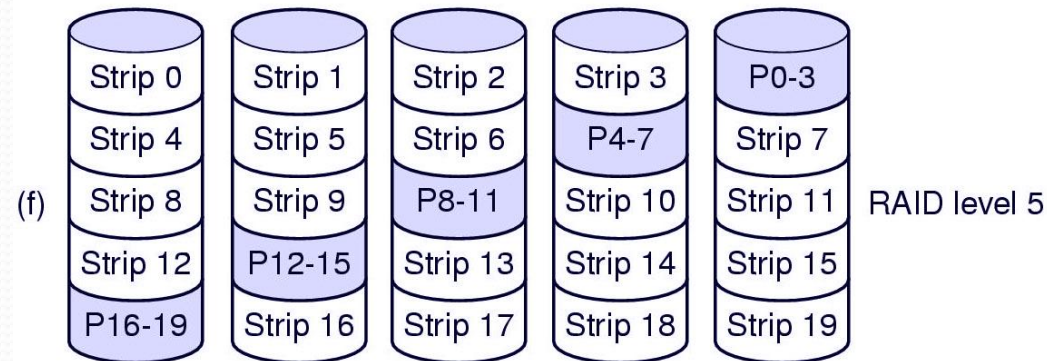
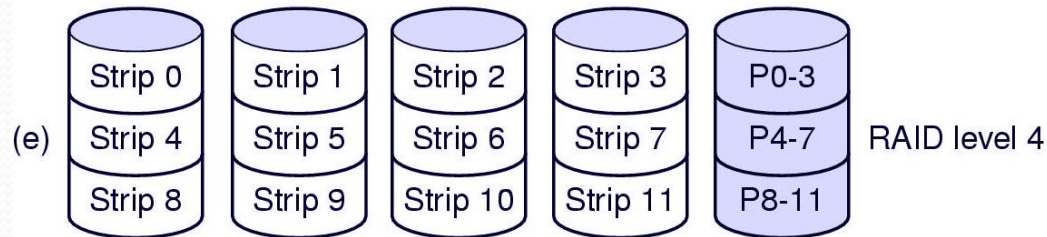
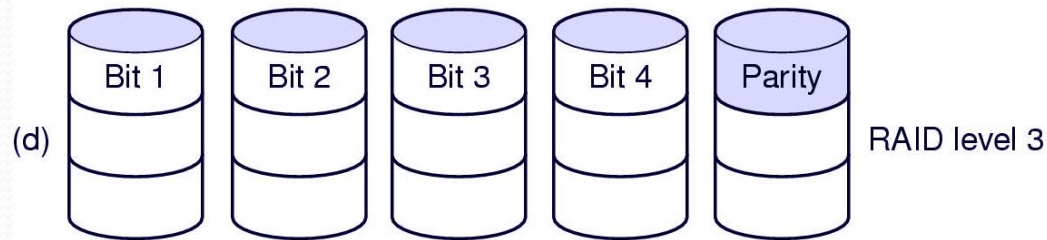
An example of Hamming code word:

- Let a byte of data be: 1 0 0 1 1 0 1 0
- First create the code word, leaving spaces for the parity bits in positions 1, 2, 4 and 8: _ _ 1 _ 0 0 1 _ 1 0 1 0
- Calculate the parity for each parity bit.
- Parity bit position 1: Check bits 1, 3, 5, 7, 9, 11. Since there are even number of 1's so it is even parity. Set first parity position to 0: 0 _ 1 _ 0 0 1 _ 1 0 1 0
- Parity bit position 2: Check bits 2, 3, 6, 7, 10, 11. It is odd parity, so set position 2 with 1: 0 1 1 _ 0 0 1 _ 1 0 1 0
- Parity bit position 4: Check bits 4, 5, 6, 7, 12. Odd parity so set position 4 to 1: 0 1 1 1 0 0 1 _ 1 0 1 0
- Parity bit position 8: Check bits 8, 9, 10, 11, 12. Even parity so set position 8 to 0: 0 1 1 1 0 0 1 0 1 0 1 0
- Hence the final code word = 0 1 1 1 0 0 1 0 1 0 1 0

How errors are corrected or recovered?

- Suppose **one** of the drives crashed (or contained bad bit) or in a network, **one** of the bit get flipped, so that we are left with the following information about the code word (Note: In case of disk failure, the RAID controller pretends that all its bits are zeroes):
0 1 1 1 0 0 1 0 1 **0** 1 0
- Now the bit needs to be recovered or corrected. The method used is to *verify each parity bit*. Doing so, we can discover that parity bits 2 and 8 are incorrect. So the bit position $2+8 = 10$ is the location of the bad bit, so make it 1.
- In general, check each parity bit and add the positions that are wrong to get the location of the bad bit.

RAID levels...



RAID level 3

- It is a simplified version of RAID level 2.
- Here a single parity bit is computed for each data word and written to a parity drive.
- As in RAID level 2, the drives must be exactly synchronized, since individual data words (bits) are spread over multiple drives.
- The parity drive provides full 1-bit error correction in case of a drive crash. When a drive crashes, the controller just pretends that all its bits are zeroes. If a word has a parity error, the bit from the dead drive must have been a 1, so it is corrected (recovered).
- **Advantage:** It is as good as RAID level 2 and is less expensive in the number of extra disks (it has only a one-disk overhead).
- **Disadvantage:**
 - Although both RAID level 2 and 3 offer very high data rates (words/sec), the number of separate I/O requests per second they can handle is no better than a single drive.
 - Expense of computing and writing the parity.

RAID level 4

- It uses block-level striping, as in RAID level 0, and in addition keeps a parity block on separate disk for corresponding blocks from N other disks.
- If one of the disk fails, the parity block can be used with the corresponding blocks from the other disks to restore the blocks of the failed disk.
- For example, if each strip is k bytes long, all the strips are ***EXCLUSIVE ORed*** together, resulting in a parity strip k bytes long. If a drive crashes, the lost bytes can be recomputed from the parity drive by reading the entire set of drives. [Example given on coming slide]
- Since multiple read accesses can proceed in parallel, the transfer rates for large reads is high. Since the data and parity can be written in parallel, the transfer rates for large writes is also high.
- **Problem:**
 - Small reads and writes cannot be performed in parallel.
 - Parity bottleneck.

Let each strip be 2 bytes long.
 Let us XOR the contents of strip 0,
 strip 1, strip 2 and strip 3 as:

$$\begin{array}{r}
 \begin{array}{cccccccccccccccc}
 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1
 \end{array} \\
 \oplus \begin{array}{cccccccccccccccc}
 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0
 \end{array} \\
 \oplus \begin{array}{cccccccccccccccc}
 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0
 \end{array} \\
 \oplus \begin{array}{cccccccccccccccc}
 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1
 \end{array} \\
 \hline
 \begin{array}{cccccccccccccccc}
 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0
 \end{array}
 \end{array}$$

This is the parity. Suppose that the
 disk containing strip 0 crashed. Then
 by XORing parity, strip 1, strip 2
 and strip 3, we can get the contents
 of strip 0.

RAID level 5

- Differs from level 4 only in the way that parity information is distributed in all $N+1$ disks, rather than storing data in N disks and parity in one disk.
- A parity block cannot store parity for blocks in the same disk, because a disk failure would result in loss of data as well as of parity, and hence would not be recoverable. The advantage is that the potential overuse of a single parity disk as in RAID 4 is avoided.

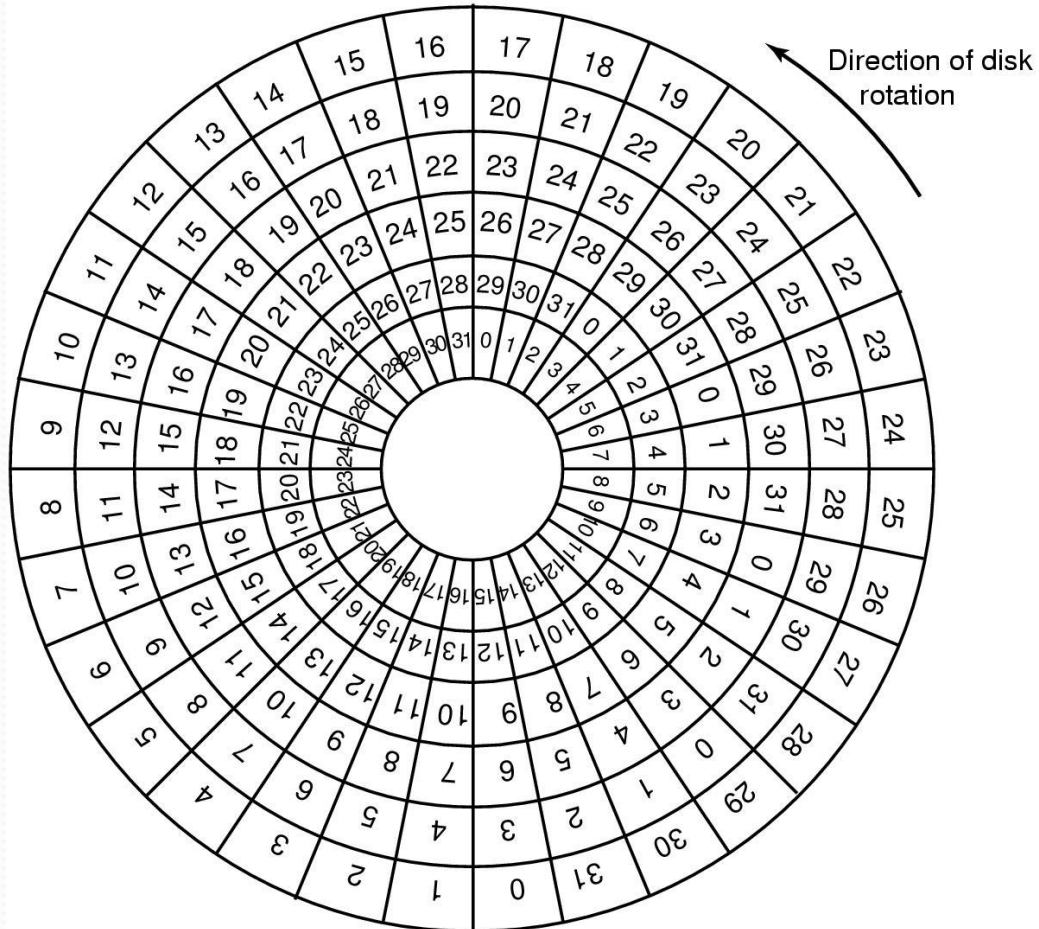
Disk Formatting

- Before a disk can store data, it must be divided into sectors that the disk controller can read and write, called *low-level formatting* (or *physical formatting*). Done by the manufacturer.
- The sector typically consists of **preamble**, **data** and **ECC** (Error Correction Code).
- The preamble contains the cylinder and sector number and the ECC contains redundant information that can be used to recover from read error.
- The size of the ECC field depends upon the manufacturer, depending on how much disk space the designer is willing to give up for higher reliability.



Fig: A disk sector

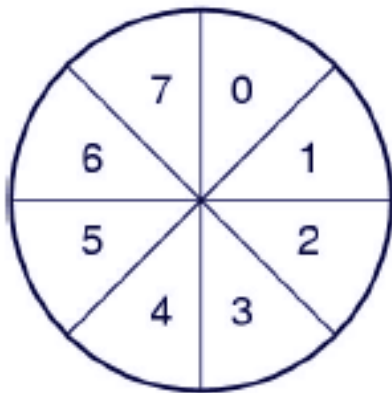
Disk Formatting...



Disk Formatting...

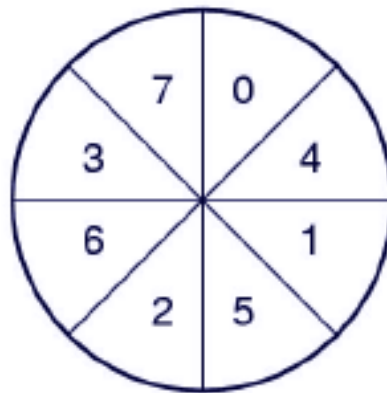
- The position of sector 0 on each track is offset from the previous track when the low-level format is laid down. This offset is called **cylinder skew** and is done to improve performance.
- For example, if disk I/O operations are limited to transferring a single sector at a time (which requires a disk controller's one-sector buffer), then to read two consecutive sectors, it reads the first sector from the disk, does the ECC calculation, and then transfers data to main memory. During this time the next sector will fly by the head because the disk is rotating. When the copy to memory is complete, the controller will have to wait almost an entire rotation for the second sector to come around again.
- This problem can be eliminated by numbering the sectors in an interleaved fashion when formatting the disk.
- In figure(a) we see the usual numbering pattern (ignoring cylinder skew). In figure(b) we see **single interleaving**, which gives the controller some time between consecutive sectors in order to copy the buffer to main memory. If the copying process is very slow, the **double interleaving** method is used.

Disk Formatting...



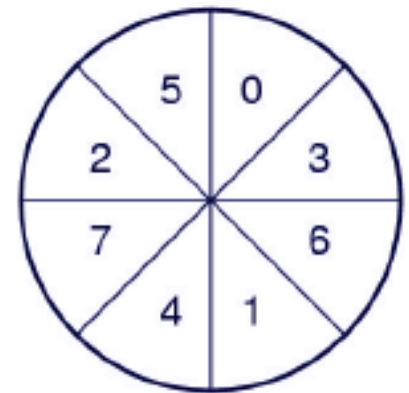
(a)

a) No interleaving



(b)

b) Single interleaving



(c)

c) Double interleaving

Disk Formatting...

- After low-level formatting is completed, the disk is partitioned. Logically, each partition is a separate disk. Partitions are needed to allow multiple OSs to coexist. The sector 0 of the disk contains the ***master boot record*** which contains some boot information plus the partition table. The partition table gives the starting sector and size of each partition. To be able to boot from the hard disk, one partition must be marked as active in the partition table.
- After partitioning, the final step in preparing a disk for use is to perform a ***high-level format*** of each partition. This operation lays down a boot block, the free storage administration (bitmap or free list), root directory, and an empty file system. It also puts a code in the partition table entry telling which file system is used in the partition because many operating systems support multiple incompatible file systems.

TU Exam Question 2068

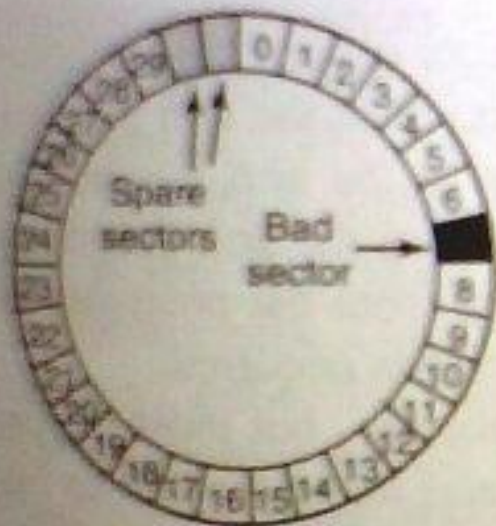
- A disk has 8 sectors per track and spins at 600 rpm. It takes the controller time 10ms from the end of one I/O operation before it can issue a subsequent one. How long does it take to read all 8 sectors using the following interleaving system?
 - (a) No interleaving
 - (b) Single interleaving
 - (c) Double interleaving

Error Handling

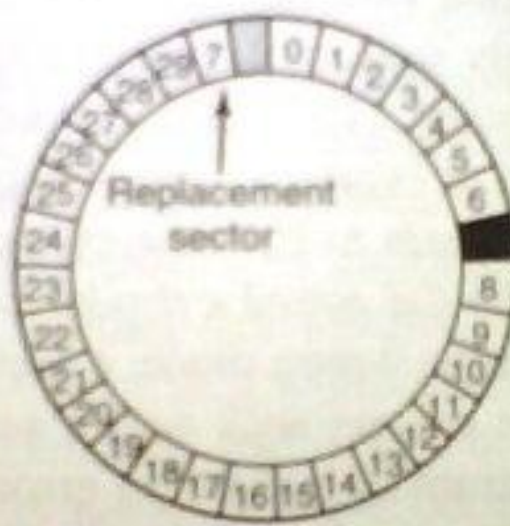
Bad Blocks/Sectors:

- Most frequently, one or more sectors become defective or most disks even come from factory with bad blocks.
- Depending on the disk and controller in use, these blocks are handled in a variety of ways.
 1. Handling bad blocks manually - For example, run MSDOS *chkdsk* command to find bad blocks, and *format* command to create new blocks. Whenever a bad block is found, the FAT entry is updated to tell the allocation routines not to use that block. Data that resided on bad blocks usually are lost.
 2. Using bad block recovery - The controller maintains the list of bad blocks on the disk and for each bad block, one of the spares is substituted.

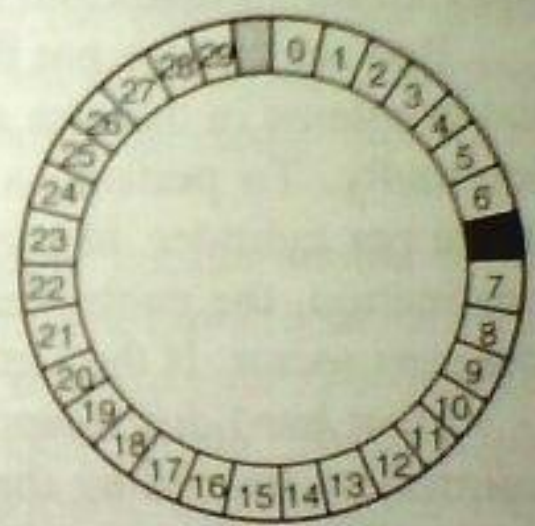
Error Handling...



(a)



(b)



(c)

Figure (a) A disk track with a bad sector. (b) Substituting a spare for the bad sector. (c) Shifting all the sectors to bypass the bad one.

TU Exam Question 2067

- Write short notes on :
 - (a) Disk scheduling algorithms
 - (b) Error handling and formatting

RAM Disks

- RAM Disk is a virtual block device created from main memory.
- Commands to read or write disk blocks are implemented by RAM disk driver.
- It completely eliminates seek and rotational delays suffered in disk devices.
- RAM disks are particularly useful for storing files that are frequently accessed or are temporary.
- RAM disks are especially used in high performance applications.
- Some OS define the RAM disks at boot time, others dynamically.
- **Disadvantages:** Cost, capacity and volatility. The volatility is solved by providing battery backups.

Stable Storage Management

- Ideally, a disk should work all the time with no errors. However, this is not achievable. What is achievable is a disk subsystem that has the following property: *when a write is issued to it, the disk either correctly writes the data or it does nothing, leaving the existing data intact*. Such a system is called **stable storage** and is implemented in software. The goal is to keep the disk consistent in all costs.
- Stable storage uses a pair of identical disks with the corresponding blocks working together to form one error-free block. In the absence of errors, the corresponding blocks on both drives are the same. Either one can be read to get the same result. To achieve this goal, the following three operations are defined:

Stable Storage Management...

1. **Stable writes.** A stable write consists of first writing the block on drive 1, then reading it back to verify that it was written correctly. If it was not written correctly, the write and reread are done again up to n times until they work. After n consecutive failures, the block is remapped onto a spare and the operation repeated until it succeeds, no matter how many spares have to be tried. After the write to drive 1 has succeeded, the corresponding block on drive 2 is written and reread, repeatedly if need be, until it, too, finally succeeds. In the absence of CPU crashes, when a stable write completes, the block has correctly been written onto both drives and verified on both of them.
2. **Stable reads.** A stable read first reads the block from drive 1. If this yields an incorrect ECC, the read is tried again, up to n times. If all of these give bad ECCs, the corresponding block is read from drive 2. Since a successful stable write leaves two good copies of the block behind, a stable read always succeeds.
3. **Crash recovery.** After a crash, a recovery program scans both disks comparing corresponding blocks. If a pair of blocks are both good and the same, nothing is done. If one of them has an ECC error, the bad block is overwritten with the corresponding good block. If a pair of blocks are both good but different, the block from drive 1 is written onto drive 2 because drive 1 was first written.

TU Exam Question 2067

- Explain the disk management with example.

Rest

- **Do it Yourself!!!**